



HUST

TRƯỜNG ĐẠI HỌC BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

ONE LOVE. ONE FUTURE.

The background of the entire image is a dark blue field filled with a pattern of red dots. These dots are arranged in a way that they form a large, stylized, abstract shape in the center, resembling a heart or a shield. The dots are of varying sizes and are more densely packed in some areas than others, creating a textured, pixelated effect.

SOICT

School of Information and Communication Technology

ONE LOVE. ONE FUTURE.



TRƯỜNG ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Software Quality Assurance and Testing

2 – Software Metrics

ONE LOVE. ONE FUTURE.

Contents of the lecture

- Motivation of Software Metrics
- Lines of code
- Halstead's metrics
- McCabe Cyclomatic Number
- Maintainability Index (MI)
- OOP Software metrics



How to achieve a successful software project?

- In order to conduct a successful software project we must understand
 - the scope of work to be done
 - the risks incurred
 - the resources required
 - the tasks to be accomplished
 - the milestones to be tracked
 - the cost
 - the schedule to be followed



Project Management

- Before a project can be planned
 - objectives and scope should be established
 - alternative solutions should be considered
 - technical and management constraints should be identified
- This information is required to estimate costs, tasks, and a schedule



Software Metrics

- Metrics help us understand the technical process that is used to develop a product
 - The process is measured to improve it and the product is measured to increase its quality
- Measuring software projects is still controversial
 - Not yet clear which are the appropriate metrics for a software project
 - whether people, processes or products can be compared using such metrics



Why use metrics?

- Without measurements there is no real way to determine if the process is improving
- They allow the establishment of meaningful goals for improvement
- Metrics allow an organization to identify the causes of defects that have the greatest effect on software development



Applying metrics

- When metrics are applied to a product they help identify
 - which user requirements are likely to change
 - which modules are most error prone
 - how much testing should be planned for each module



Direct measurements

- Measurements can be either direct or indirect
- Direct measures are taken from a feature of an item (e.g., length)
 - Lines of code, execution speed, memory size, defects reported



Indirect Measurements

- Indirect measurements associate a measure to a feature of the object being measured (e.g., quality is based upon counting rejects)
 - functionality, quality, complexity, efficiency, reliability and maintainability



Code Complexity Measurements

Các công cụ phân tích mã nguồn

- Code complexity correlates with the defect rate and robustness of the application program
- Code complexity measurement tools
 - Testwell CMT++ for C, C++ and C#
 - Testwell CMTJava for Java
- Code with good complexity
 - contains less errors
 - is easier to understand
 - is easier to maintain



How to use code complexity measurements

- Code complexity metrics are used to locate complex code
- To obtain a high quality software with low cost of testing and maintenance, the code complexity should be measured as early as possible in coding --> developers can adapt their code when recommended values are exceeded



Code complexity metrics

- Metrics shown by Testwell CMT++/CMTJava
 - Lines of code metrics
 - McCabe Cyclomatic number
 - Halstead Metrics
 - Maintainability Index



Lines of code metrics

- Most traditional measures used to quantify software complexity
- Simple, easy to count and very easy to understand
- Do not take into account the intelligence content and the layout of the code
- Testwell CMT++/CMTJava calculates the following lines of code metrics
 - LOCphy: number of physical lines
 - LOCbl: number of blank lines (a blank line inside a comment block is considered to be a comment line)
 - LOCpro: number of program lines (declarations, definitions, directives and code)
 - LOCcom: number of comment lines



Lines of code metrics - Recommendations

Những giá trị khuyên dùng cho chiều dài mã nguồn

• Function length

- Function length should be 4 to 40 program lines
- A function definition contains at least a prototype, one line of code, a pair of braces, which makes 4 lines
- A function longer than 40 program lines probably implements many functions
- Exception: functions containing one selection statement with many branches) --> decomposing them into smaller functions often decreases readability





- **File length**

- File length should be 4 to 400 program lines
- The smallest entity that may reasonably occupy a whole source file is a function and the minimum length of a function is 4 lines
- Files longer than 400 program lines (10..40 functions) are usually too long to be understood as a whole



- **Comments**

- At least 30% and at most 75% of a file should be comments
- If less than one third of a file is comments the file is either very trivial or poorly explained
- If more than 75% of a file are comments, the file is not a program but a document
- Exception: in a well-documented header file percentage of comments may sometimes exceed 75%

Halstead's Metrics

- Developed by Maurice Halstead
- Introduced in 1977
- Used and experimented extensively since that time
- One of the oldest measures of program complexity
- Strong indicators of code complexity
- Often used as a maintenance metric



Halstead's Metrics (cont.)

- Based on interpreting the source code as a sequence of tokens and classifying each token to be an operator or an operand
- **Operator: toán tử – Operand: toán hạng**
- Then is counted
 - number of unique (distinct) operators (n_1)
 - number of unique (distinct) operands (n_2)
 - total number of operators (N_1)
 - total number of operands (N_2)
- All other Halstead measures are derived from these four quantities with certain fixed formulas as described later



Operators and Operands

- Operators
 - traditional: +, -, *, /, ++, --, etc.
 - keywords: return, if, continue, break, try, catch etc.
- Operands
 - identifiers
 - constants



Halstead's Metrics

- **Program length (N):** the program length is the sum of the total number of operators and operands in the program:
 - $N = N_1 + N_2$
- **Vocabulary size (n):** the vocabulary size is the sum of the number of unique operators and operands
 - $n = n_1 + n_2$
- **Program volume (V):** information content of the program
 - $V = N * \log_2(n)$
- **Halstead's volume** describes the size of the implementation of an algorithm



Halstead's metrics - Recommendations

- The volume of a function should be at least 20 and at most 1000
 - The volume of a parameterless one-line function that is not empty is about 20
 - A volume greater than 1000 tells that the function probably does too many things
 - The volume of a file should be at least 100 and at most 8000
 - These limits are based on volumes measured for files whose LOCpro and $v(G)$ are near their recommended limits



Other Halstead's metrics

- **Difficulty level (D)**

- The difficulty level or error proneness (D) of the program is proportional to the number of unique operator in the program
- D is also proportional to the ration between the total number of operands and the number of unique operands
 - i.e., if the same operands are used many times in the program, it is more prone to errors
- $D = (n1/2) * (N2 / n2)$

- **Program level (L)**

- The program level (L) is the inverse of the error proneness of the program
- i.e., A low level program is more prone to errors than a high level program
- $L = 1 / D$



Other Halstead's metrics

- **Effort to implement (E)**

- The effort to implement or understand a program is proportional to the volume and to the difficulty level of the program

- $E = V * D$

- **Time to implement (T)**

- The time to implement or understand a program (T) is proportional to the effort
- Halstead has found that dividing the effort by 18 give an approximation for the time in seconds

- $T = E / 18$



Other Halstead's metrics

- **Number of delivered bugs (B)**

- The number of delivered bugs (B) correlates with the overall complexity of the software
- $B = (E^2/3)/3000$
- Estimates for the number of errors in the implementation
- Delivered bugs in a file should be less than 2
- Experiences have shown that when programming with C or C++, a source file almost always contains more errors than B suggests
- B is an important metric for dynamic testing
- The number of delivered bugs approximates the number of errors in a module
- As a goal at least that many errors should be found from the module in its testing



Halstead's metric Example

Ví dụ về độ đo Halstead

```
void sort ( int *a, int n ) {  
    int i, j, t;  
  
    if ( n < 2 ) return;  
    for ( i=0 ; i < n-1; i++ ) {  
        for ( j=i+1 ; j < n ; j++ ) {  
            if ( a[i] > a[j] ) {  
                t = a[i];  
                a[i] = a[j];  
                a[j] = t;  
            }  
        }  
    }  
}
```

$V = 80 \log_2(24) \approx 392$



Halstead Example (cont.)

- Step 1:

- List all the operands and operators in the source file
- Count the number of distinct operands and distinct operators

- Step 2:

- Calculate V, D, E...

```
void sort ( int *a, int n ) {  
    int i, j, t;  
    if ( n < 2 ) return;  
    for ( i=0 ; i < n-1; i++ ) {  
        for ( j=i+1 ; j < n ; j++ ) {  
            if ( a[i] > a[j] ) {  
                t = a[i];  
                a[i] = a[j];  
                a[j] = t;  
            }  
        }  
    }  
}
```

$V = 80 \log_2(24) \approx 392$

- Ignore the function definition
- Count operators and operands

3	<	3	{	1	0
5	=	3	}	2	1
1	>	1	+	1	2
1	-	2	++	6	a
2	,	2	for	8	i
9	;	2	if	7	j
4	(	1	int	3	n
4	)	1	return	3	t
6	[]				

	Total	Unique
Operators	N1 = 50	n1 = 17
Operands	N2 = 30	n2 = 7



McCabe Cyclomatic Number

Chỉ số độ phức tạp McCabe

- The cyclomatic complexity $v(G)$ has been introduced by Thomas McCabe in 1976
- Measures the number of linearly-independent paths through a program module (Control Flow)
- The McCabe complexity is one of the more widely-accepted software metrics, it is intended to be independent of language and language format
- Considered as a broad measure of soundness and confidence for a program



McCabe Cyclomatic Number – $v(G)$

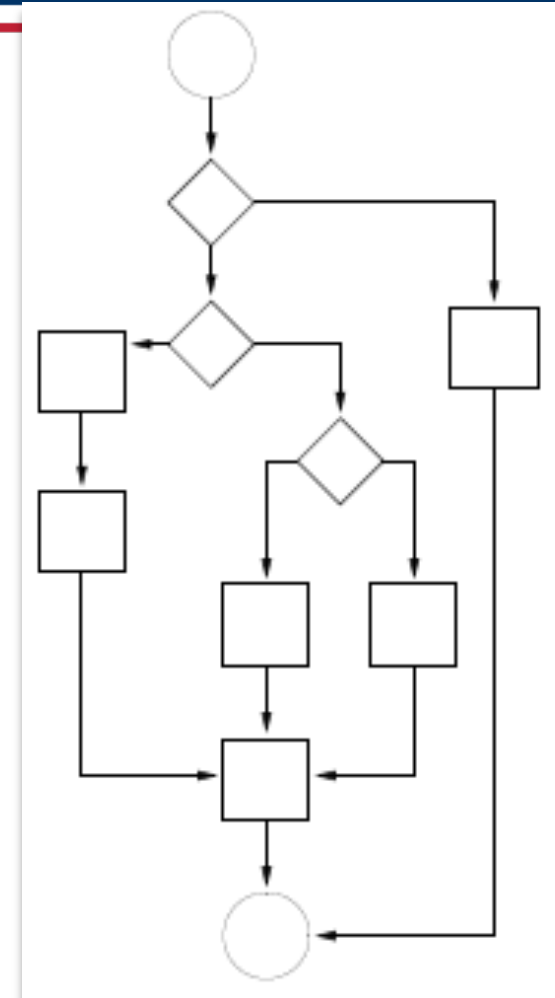
- $v(G)$ is the number of conditional branches
- $v(G) = 1$ for a program consisting of only sequential statements
- For a single function, $v(G)$ is one less than the number of conditional branching points in the function
- The greater the cyclomatic number is, the more execution paths there are through the function, and the harder it is to understand



McCabe Cyclomatic Number

Công thức tính chỉ số McCabe

- Describes the structural complexity
 - Control flow
 - Data flow
- Graph based metrics
 - Number of vertices
 - Number of edges
 - Maximum length (depth of graph)
- In general: $v(G) = \#edges - \#vertices + 2$
- For control flow graphs
 - $v(G) = \#binaryDecision + 1$
 - $v(G) = \#IFs + \#LOOPs + 1$



McCabe Cyclomatic Number

- For dynamic testing, the cyclomatic number $v(G)$ is one of the most important complexity measures
- Because the cyclomatic complexity describes the control flow complexity
 - it is obvious that modules and functions having high cyclomatic number need more test cases than modules having a lower cyclomatic number
 - Rule: each function should have at least as many test cases as indicated by its cyclomatic number



McCabe Cyclomatic Number - Example

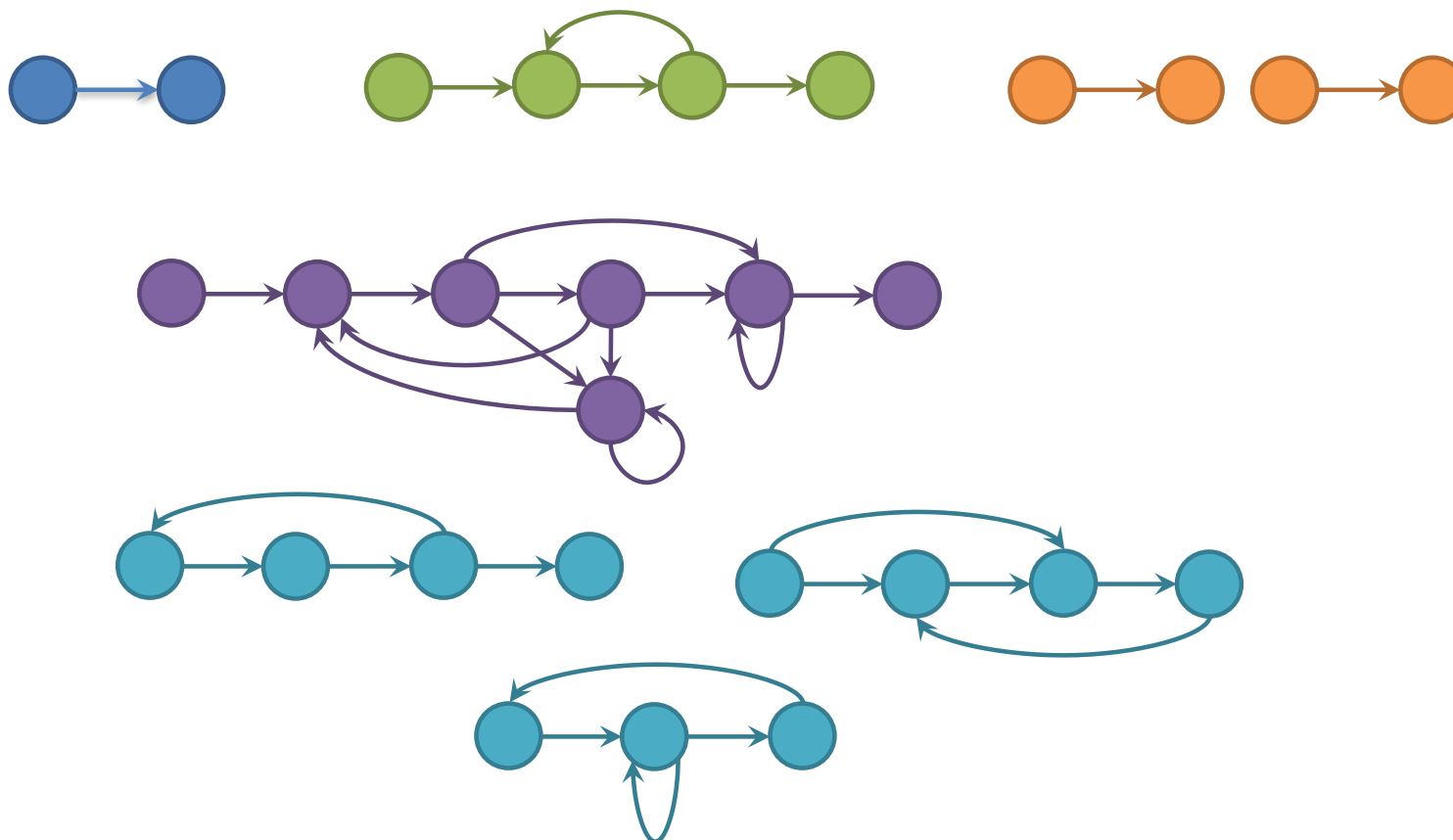
- Count IFs and LOOPS
- IF: 2, LOOP: 2
- $v(G) = 5$

```
void sort ( int *a, int n ) {  
    int i, j, t;  
  
    if ( n < 2 ) return;  
    for ( i=0 ; i < n-1; i++ ) {  
        for ( j=i+1 ; j < n ; j++ ) {  
            if ( a[i] > a[j] ) {  
                t = a[i];  
                a[i] = a[j];  
                a[j] = t;  
            }  
        }  
    }  
}
```

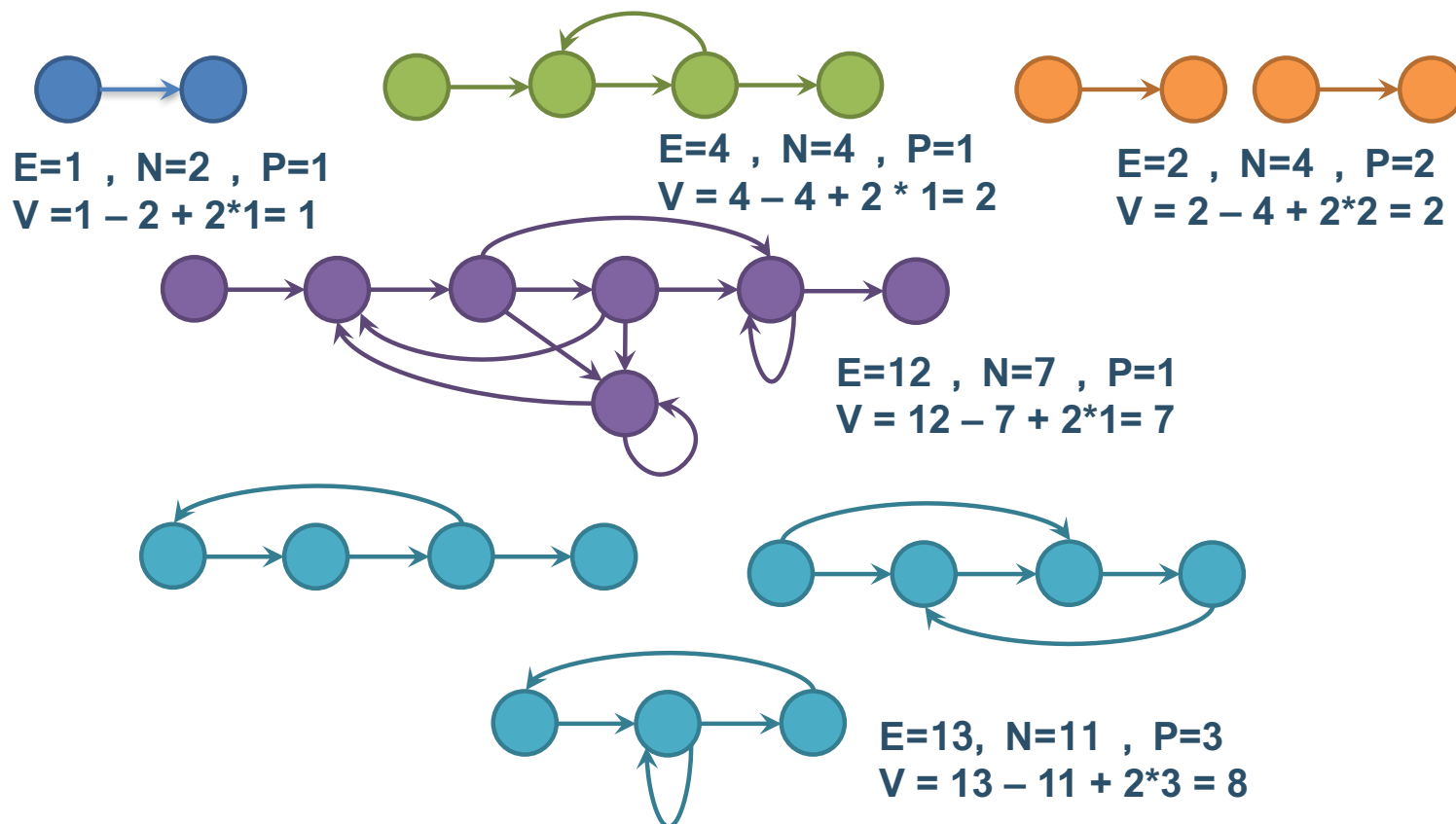
$$V = 80 \log_2(24) \approx 392$$



Other examples based on CFG



Other examples based on CFG



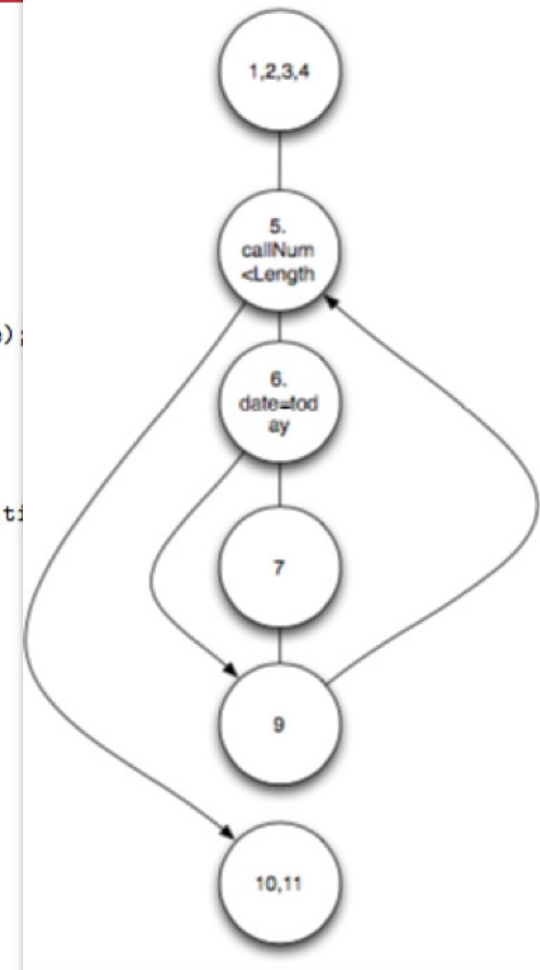
Other examples based on source code

```
1 procedure SpeakingTimeToday (in listOfCalls; out speakingTime);
2 begin
3   callNum = 0
4   speakingTime = 0
5   while (callNum < listOfCalls.Length)
6     if listOfCalls[callNum].date = today
7       speakingTime = speakingTime + listOfCalls[callNum].time
8     end
9     callNum = callNum + 1
10  end
11 end
```



Other examples based on source code

```
1 procedure SpeakingTimeToday (in listOfCalls; out speakingTime);
2 begin
3   callNum = 0
4   speakingTime = 0
5   while (callNum < listOfCalls.Length)
6     if listOfCalls[callNum].date = today
7       speakingTime = speakingTime + listOfCalls[callNum].t
8     end
9     callNum = callNum + 1
10  end
11 end
```



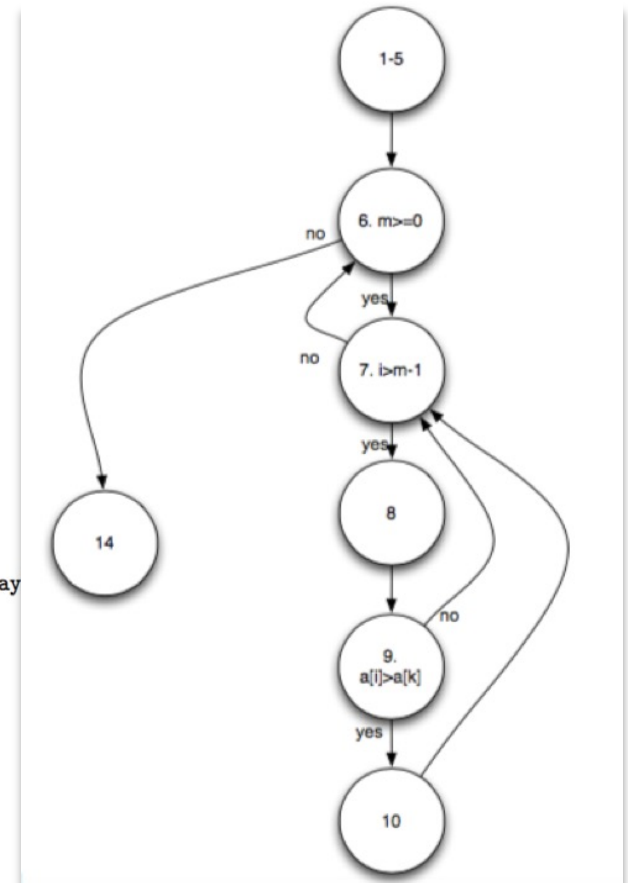
Other examples based on source code

```
1 public class MyBubbleSort {
2
3     public static void bubble_srt(int array[]) {
4         int n = array.length;
5         int k;
6         for (int m = n; m >= 0; m--) {
7             for (int i = 0; i < m - 1; i++) {
8                 k = i + 1;
9                 if (array[i] > array[k]) {
10                     swapNumbers(i, k, array);
11                 }
12             }
13         }
14     }
15
16     private static void swapNumbers(int i, int j, int[] array) {
17         int temp;
18         temp = array[i];
19         array[i] = array[j];
20         array[j] = temp;
21     }
22 }
```



Other examples based on source code

```
1 public class MyBubbleSort {
2
3     public static void bubble_srt(int array[]) {
4         int n = array.length;
5         int k;
6         for (int m = n; m >= 0; m--) {
7             for (int i = 0; i < m - 1; i++) {
8                 k = i + 1;
9                 if (array[i] > array[k]) {
10                     swapNumbers(i, k, array);
11                 }
12             }
13         }
14     }
15
16     private static void swapNumbers(int i, int j, int[] array) {
17         int temp;
18         temp = array[i];
19         array[i] = array[j];
20         array[j] = temp;
21     }
22 }
```



McCabe Cyclomatic Number – Recommendations

- The cyclomatic number of a function should be less than 15.
 - i.e., if a function has a cyclomatic number of 15, there are at least 15 (but probably more) execution paths through it
- More than 15 paths are hard to identify and test
 - Functions containing one selection statement with many branches make up an exception
- A reasonable upper limit cyclomatic number of a file is 100



Maintainability Index (MI)

Chỉ số bảo trì mã nguồn

- Calculated with certain formulas from lines of code measures, McCabe measure and Halstead measures
- Indicates when it becomes cheaper and/or less risky to rewrite the code instead to change it
- Two variants of Maintainability Index:
 - One that contains comments (MI) and one that does not contain comments (MIwoc)
- Actually there are three measures:
 - MIwoc: Maintainability Index without comments
 - Mlcw: Maintainability Index comment weight
 - MI: Maintainability Index = **MIwoc + Mlcw**



MI's formula

Công thức tính MI

- $MI_{woc} = 171 - 5.2 * \ln(aveV) - 0.23 * aveG - 16.2 * \ln(aveLOC)$
 - $aveV$ = average Halstead Volume V per module
 - $aveG$ = average extended cyclomatic complexity $v(G)$ per module
 - $aveLOC$ = average count of lines LOC_{phy} per module
 - $perCM$ = average percent of lines of comments per module
- $MI_{cw} = 50 * \sin(\sqrt{2,4 * perCM})$



MI Recommendations

Cách tính và sử dụng chỉ số MI

- Maintainability Index (MI, with comments) values:
 - 85 and more: good maintainability
 - 65-85: Moderate maintainability
 - < 65: Difficult to maintain with really bad pieces of code
 - big, uncommented, unstructured module can lead to a MI value can be event negative



Maintainability Index Example

- Halstead's $V = 392$
- McCabe's $v(G) = 5$
- LOC = 14
- MI = 96 --> easy to maintain!

```
void sort ( int *a, int n ) {  
    int i, j, t;  
  
    if ( n < 2 ) return;  
    for ( i=0 ; i < n-1; i++ ) {  
        for ( j=i+1 ; j < n ; j++ ) {  
            if ( a[i] > a[j] ) {  
                t = a[i];  
                a[i] = a[j];  
                a[j] = t;  
            }  
        }  
    }  
}
```

$$V = 80 \log_2(24) \approx 392$$



Exercise: Calculate the MI

1. Draw the control flow graph
2. Calculate McCabe Cyclomatic
3. Calculate MI

```
1 public class BinarySearch {  
2     public int binsearch(int x, int[] V, int n) {  
3         int low, high, mid;  
4         low = 0;  
5         high = n - 1;  
6         while (low <= high) {  
7             mid = (low + high) / 2;  
8             if (x < V[mid])  
9                 high = mid - 1;  
10            else if (x > V[mid])  
11                low = mid + 1;  
12            else  
13                return mid;  
14        }  
15        return -1;  
16    }  
17 }
```



OOP Software Metrics



Objectives

- Primary objectives for object oriented metrics are no different than those for metrics used for conventional software:
- To better understand the quality of the product
- To assess the effectiveness of the process
- To improve the quality of work performed at a project level



Characteristics of object oriented metrics

- Metrics for OO systems must be tuned to the characteristics that distinguish OO from conventional software
- So there are five characteristics that lead to specialized metrics:
 - Localization
 - Encapsulation
 - Information hiding
 - Inheritance
 - Object abstraction



Localization

- Localization is a characteristic of software that indicates the manner in which information is concentrated within a program
- In the OO context, information is concentrated by summarize both data and process within the bounds of a class or object
- Since the class is the basis unit of an OO system, localization is based on objects
- Therefore metrics should apply to the class as a complete entity



Encapsulation

- Defines encapsulation as “the **packaging** (or binding together) of a collection of items
 - For example, at low level of code, encapsulation includes records and arrays
 - For OO programs, encapsulation includes the responsibilities of a class including its attributes and operations and the states of the class as defined by specific attribute values
- Encapsulation influences metrics by changing the focus of measurement from a single module to a **package of data** (attributes) and **processing modules** (operations)



Information hiding

- Information hiding suppresses (or hides) the operational details of a program component
- Only the information necessary to access the component is provided to those other components that wish to access it
- A well-designed OO system should encourage information hiding
- An indication of the quality of the OO design



Inheritance

- Inheritance is a mechanism that enables the responsibilities of one object to be propagated to other objects
- Inheritance occurs throughout all levels of a class hierarchy
- Because inheritance is a crucial characteristic in many OO systems, many OO metrics focus on it



Abstraction

- Abstraction focuses on the essential details of a program component (either data or process) with little concern for lower level details
- Abstraction is a relative concept. As we move to higher levels of abstraction, we ignore more and more details
- Because a class is an abstraction that can be viewed at many different levels of detail and in a number of different ways (e.g., a list of operations, a sequence of states, a series of collaborations etc.)
 - OO metrics represent abstraction in terms of measures of a class



Class oriented metrics

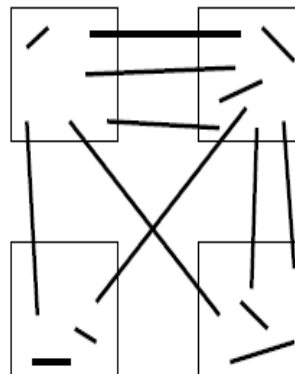
- To measure class OO metrics:
- Chidamber and Kemerer (CK) metrics suite
- Lorenz and Kidd (LK) metrics suite
- The Metrics for Object-Oriented Design (MOOD) Metrics Suite



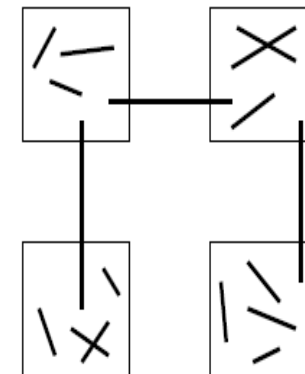
What about modularity?

- Generally, modularity is the degree to which a system may be separated and recombined
- High cohesion and Low coupling is a good design

Design A



Design B



- **Cohesion:** calls inside the module
- **Coupling:** calls between the modules

	A	B
Cohesion	Lo	Hi
Coupling	Hi	Lo

- Squares are modules, lines are calls, ends of the lines are functions.
- Which design is better?

Modularity metrics: Fan-in and Fan-out

Chỉ số về tính mô đun

- Fan-in of M: number of modules calling functions in M
- Fan-out of M: number of modules called by M
- Modules with fan-in = 0
 - deadcode
 - outside of the system boundaries
- Henry and Kafura's information flow complexity
- Fan-in and fan-out can be defined for procedures and functions
 - HK: take global data structures into account:
 - fan-in: flow of information into procedures
 - fan-out: flow of information out of procedures
 - $HK = SLOC \times (\text{fan-in} \times \text{fan-out})^2$
 - SLOC: the length (in lines of code) of the procedure without comments and blank lines



Chidamber and Kemerer's metrics

~~Đâu là Chidamber và Kemerer~~

- Weighted methods per class (WMC)
- Response for a class (RFC)
- Lack of cohesion of methods (LCOM)
- Coupling between objects (CBO)
- Depth of inheritance tree (DIT)
- Number of children (NOC)



Weighted Methods per Class (WMC)

Phân tích cấu trúc

- WMC is a count of the methods implemented within a class or the sum of the complexities of the methods
 - method complexity is measured by cyclomatic complexity
- The number of methods and their complexity is a predictor of how much time and effort is required to develop and maintain the class
- The larger the number of methods in class, the greater the potential impact on children
 - children inherit all of the methods defined in the parent class
- Classes with large number of methods are likely to be more application specific, limiting the possibility of reuse



WMC formula

```
Class A
{
  Method0 (); // complexity0
  Method1 (); // complexity1
  .
  .
  Methodn (); // complexityn
}
```

$$WMC = \sum_{i=0}^n complexity_{method}[i]$$


WMC - Example

```
public class WMPC {  
    public void one() {  
        if(true) {  
            two();  
        } else {  
        }  
        if(true && !false) {  
        }  
    }  
    public void two() {  
        if(true) {}  
    }  
    public void three(int i) {}  
}
```



Response for a Class (RFC)

Phản hồi của một lớp

- RFC is the number of distinct methods and constructors that may be invoked in response to a class's actions
 - This includes all methods accessible within the class hierarchy
- The number of methods belonging to a class and its parent, as well as those called on other classes,
- RFC indicates a degree of complexity for the class



Response for a Class (2)

- To find RFC:
 - Determine all external methods called by class methods
 - Take the union of that set and the set of class methods, called the response set (RS)
 - RFC is simply the size of RS
- $RS = M_c \cup M_e$, $RFC = |RS|$
 - M_c is the set of class methods
 - M_e is the set of external methods called by class methods



Example

Class: ObjectList

Method:	Insert	Remove	Move	isEmpty	Clear	Find	Show
External Calls:	0	~object	move	0	0	equals	draw

RS = {Insert, Remove, Move, isEmpty, Clear, Find, Show, ~object, move, equals, draw}



Response for a Class(3)

- RFC may be seen as a measure of object cohesion and coupling (or lack of)
- High RFC could indicate high coupling or low cohesion
- Low RFC may indicate opposite
- High RFC may lead to:
 - More extensive test sets
 - More difficulty in understanding class
 - Less reusability
 - More time and effort needed to fix



Lack of cohesion (LCOM)

(Sự thiếu gắn kết)

- LCOM measures the dissimilarity of methods in a class by instance variables or attributes
- Connected methods are methods that refer to a same instance variables
- Show cohesion: operate on same data
- $LCOM(C) = P - Q$ if $P > Q$ and $= 0$ if otherwise
 - P: #pairs of distinct methods in C that do not share variables
 - Q: #pairs of distinct methods in C that share variables



Lack of Cohesion in Methods (LCOM) (2)

- A highly cohesive module should stand alone
 - High cohesion indicates
 - a good class subdivision
 - simplicity and high reusability
- Lack of cohesion or low cohesion increases complexity, thereby increasing the likelihood of errors during the development process
- Classes with low cohesion could probably be subdivided into two or more subclasses with increased cohesion



Lack of Cohesion in Methods (LCOM) (3)

- Class C with M1, M2, M3
- $M1 = \{a, b, c, d, e\}$
- $M2 = \{a, b, e\}$
- $M3 = \{x, y, z\}$
- $P = \{(M1, M3), (M2, M3)\}$
- $Q = \{(M1, M2)\}$
- $LCOM = 1$



Coupling between object classes (CBO)

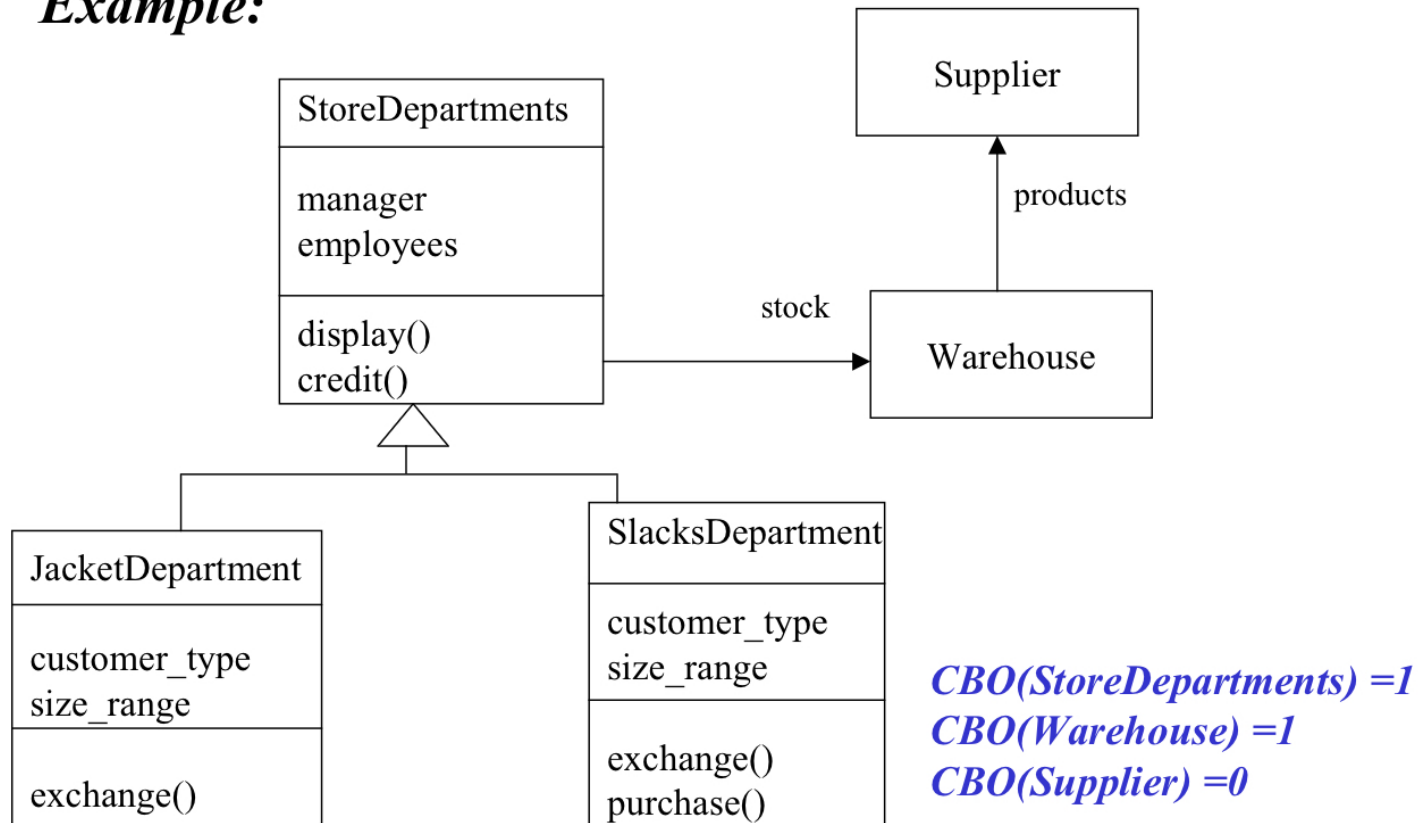
~~Cấu trúc mã nguồn tốt hơn hay xấu hơn?~~

- CBO is a count of the number of other classes to which a class is coupled
 - measured by counting the number of distinct non-inheritance related class hierarchies on which a class depends
- Goal: Lower CBO is better
 - To promote modular design
 - Reuse



Example

Example:





Coupling between object classes (CBO)

- Excessive coupling is detrimental to modular design and prevents reuse
 - the more independent class is, the easier it is reused in another application
- The larger the number of couples, the higher the sensitivity to changes in other parts of the design
 - therefore maintenance is more difficult
- Strong coupling complicates a system since a class is harder to understand, change or correct
- Complexity can be reduced by designing systems with the weakest possible coupling between classes
 - This improves modularity and promotes encapsulation



Depth of Inheritance Tree (DIT)

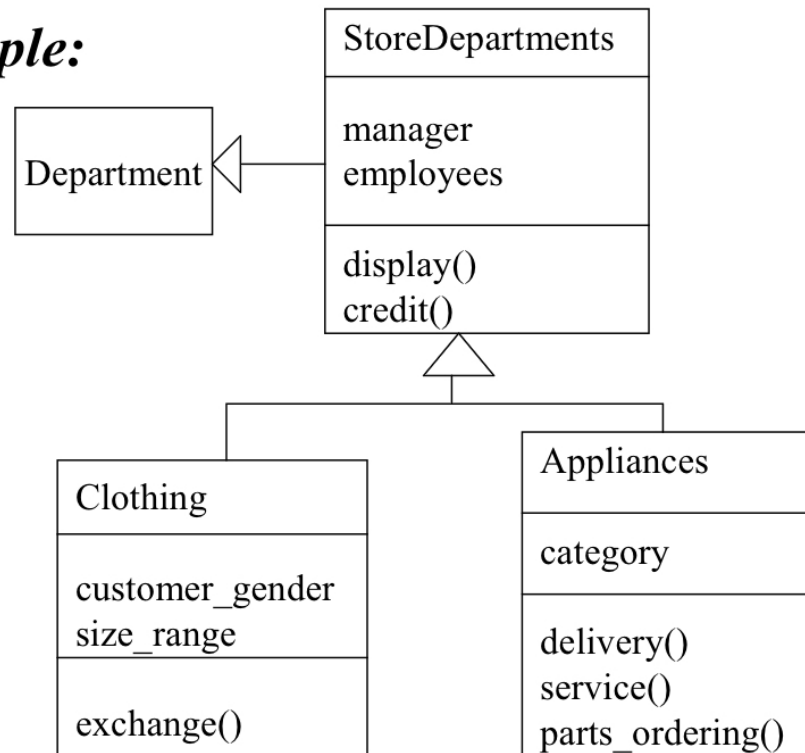
~~Độ sâu kế thừa~~

- The depth of a class within the inheritance hierarchy is the maximum number of steps from the class node to the root of the tree and is measured by the number of ancestor classes
- The deeper a class is within the hierarchy, the greater the number of methods it is likely to inherit making it more complex to predict its behavior
- Deeper trees constitute greater design complexity
 - More methods and classes are involved
 - But the greater the potential for reuse of inherited methods



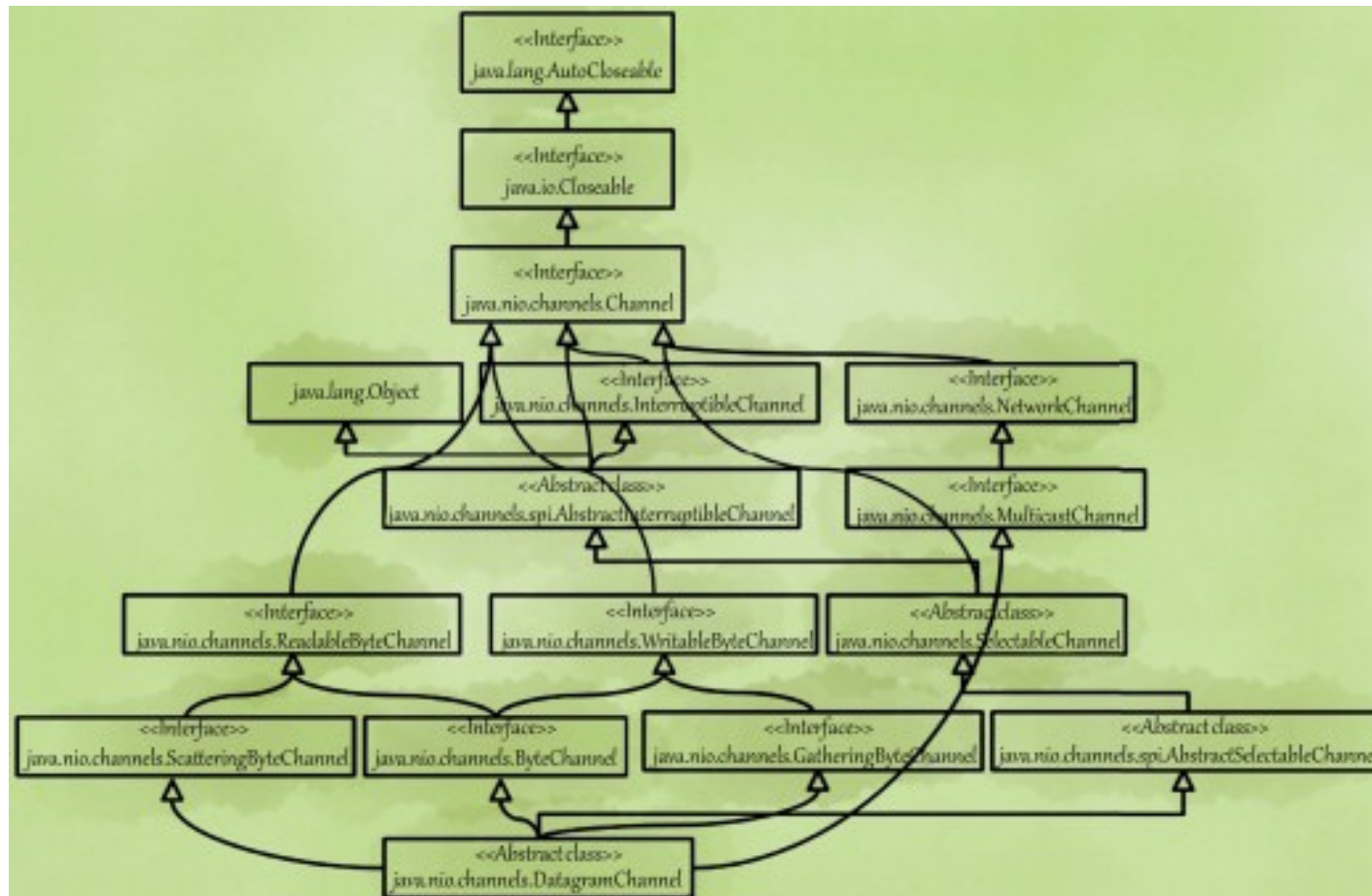
Depth of Inheritance Tree - Example

Example:



DIT (Appliances) =2
DIT(StoreDepartments)=1
DIT(Department)=0

DIT - Example

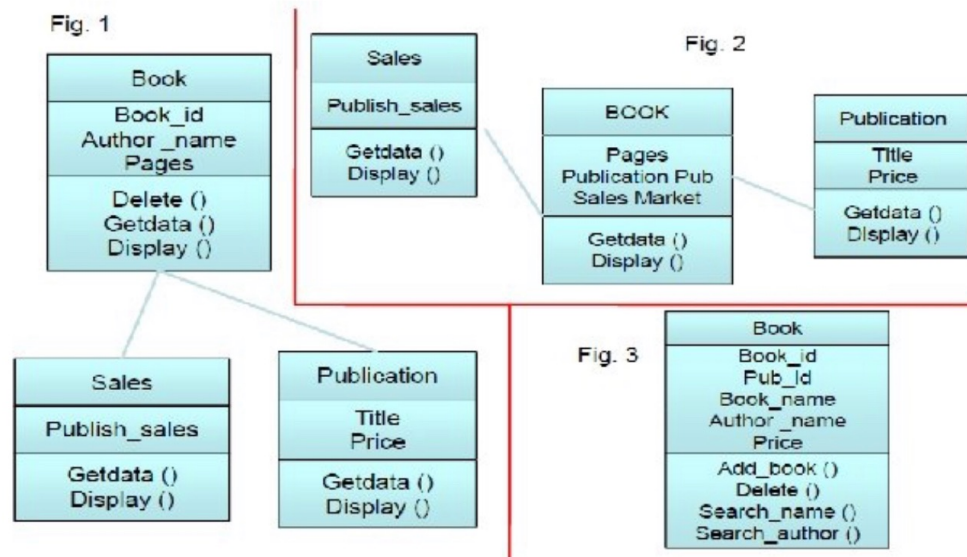


Number of Children (NOC)

- The number of children is the number of immediate subclasses subordinate to a class in the hierarchy
- If a class has a large number of children, it may require more testing of the methods of that class, thus increasing the testing time
- NOC is an indicator of the potential influence a class can have on the system
 - The greater the NOC, the greater the likelihood of improper abstraction of the parent
 - But the greater the NOC, the greater the reuse since inheritance is a form of reuse



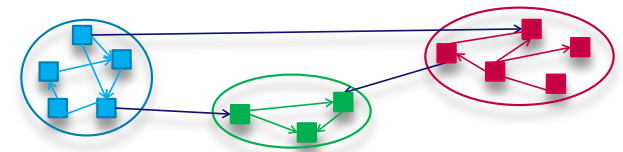
Exercise



Package-level Metrics

Các độ đo ở mức gói

- Proposed by Robert Cecil Martin (Uncle Bob)
- Focus on the relationship between packages in the project
- Martin's metrics include
 - Efferent Coupling (Ce)
 - Afferent Coupling (Ca)
 - Instability (I)
 - Abstractness (A)
 - Normalized Distance from Main Sequence (D)



Why Package?

- Grouping code into packages provides two benefits:
- Package coupling
 - It allows us to reason about the design at a higher level of abstraction
- Package cohesion
 - It helps us manage the development and distribution of the software



Case Study

Executable

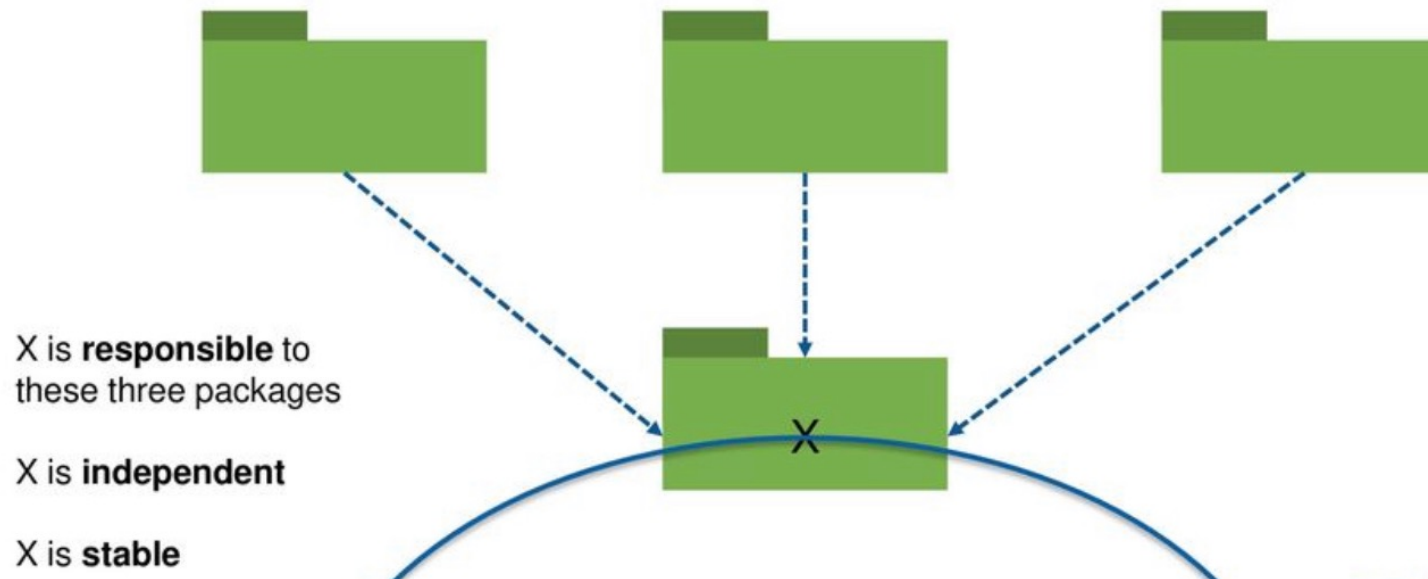
Instr Meas



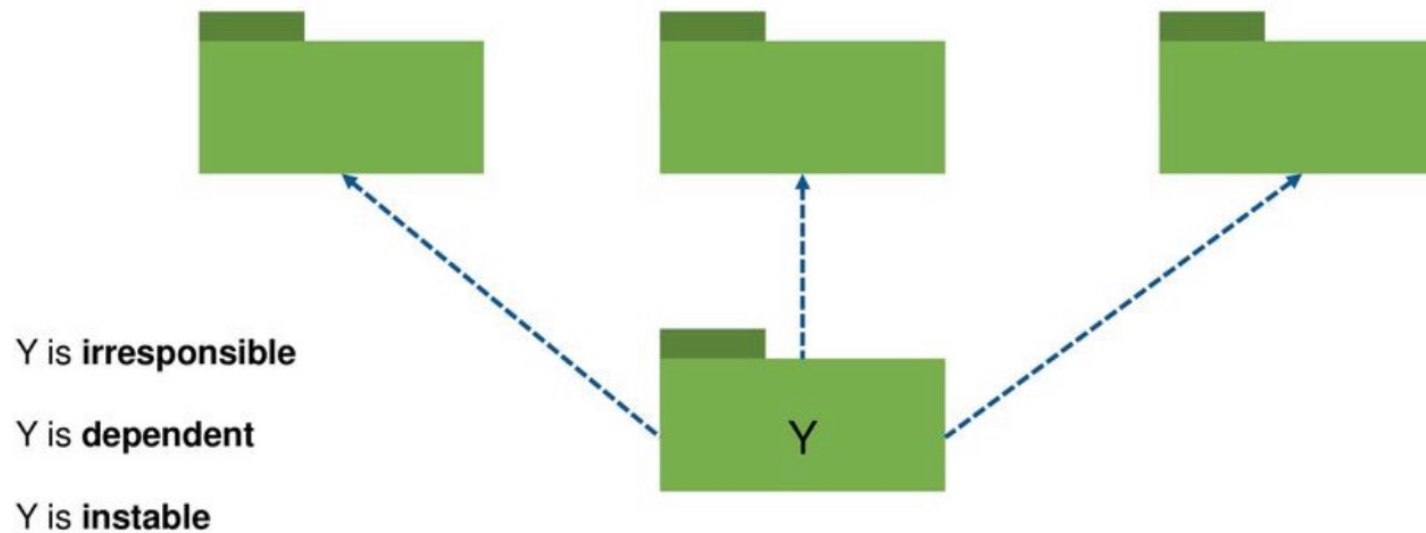
Case study



Stable Package

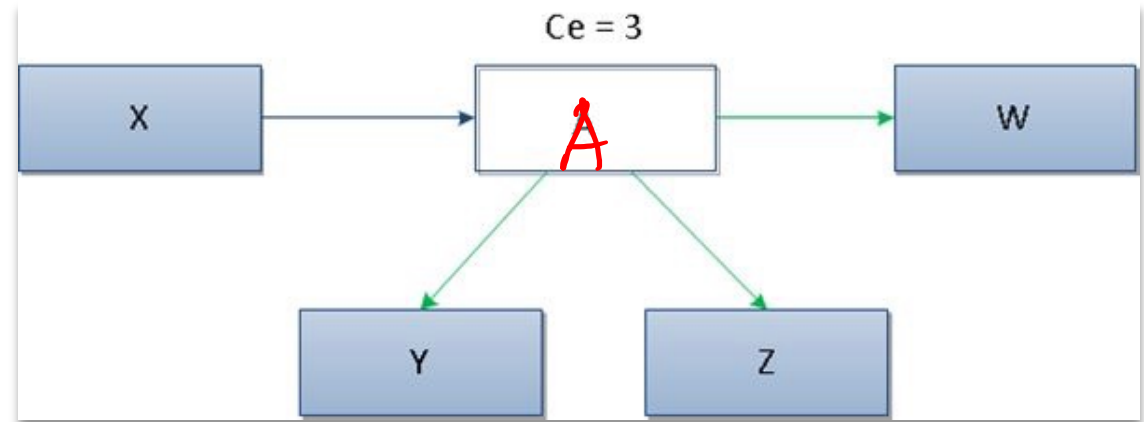


Instable Package



Efferent Coupling (CE)

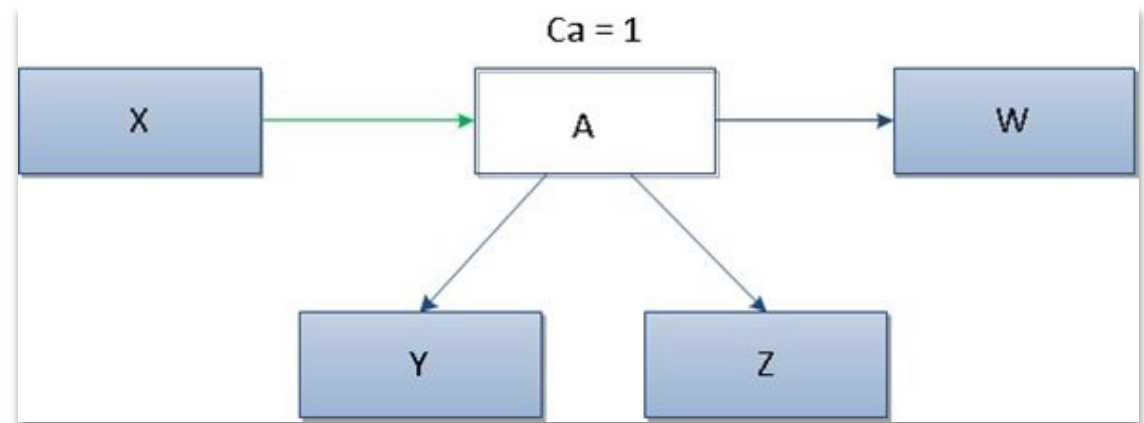
- This metric is used to measure interrelationship between classes (outgoing dependencies)
- It is a count for the number of classes in a given package which depend on the classes of other packages
- An indicator of the package's responsibility



- The high value of Ce indicates instability of a package
 - change in any of the numerous external classes can cause the need for changes to the package
- Preferred values for Ce are in the range of 0 to 20

Afferent Coupling (CA)

- Another type of dependencies between packages, i.e., incoming dependencies
- It measures the number of classes and interfaces from other packages that depend on classes in the analysed package

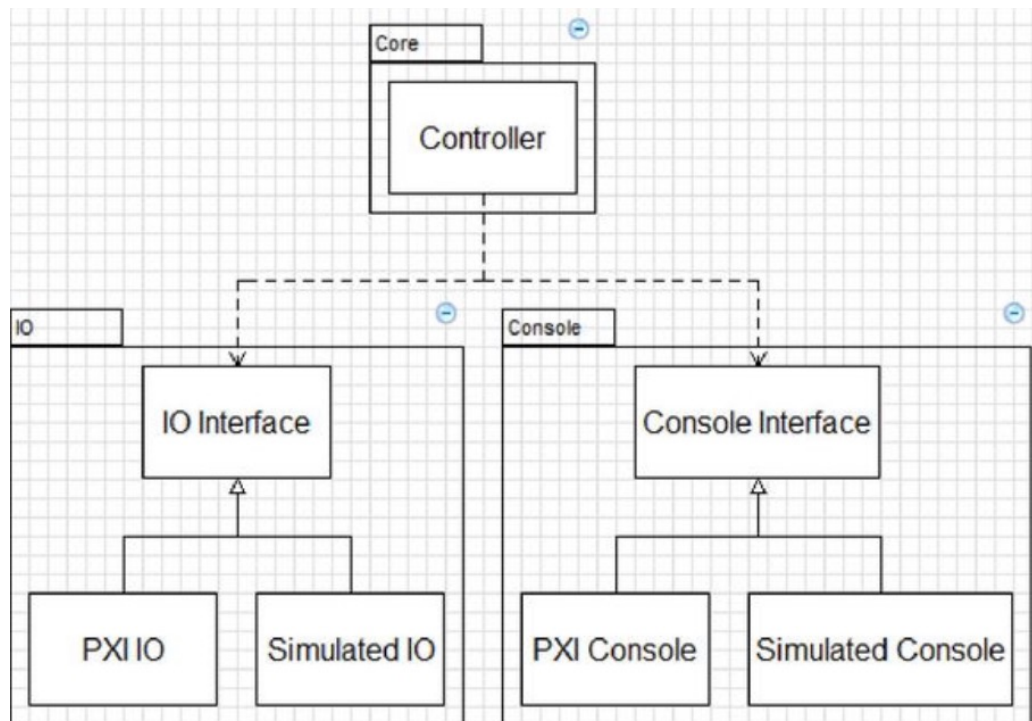


Instability (I)

- This metric is used to measure the relative susceptibility of class to changes
- $I = Ce / (Ce + Ca)$
- On the basis of value of I, we can distinguish two types of components
 - The ones having many outgoing dependencies and not many of incoming ones (value of I is close to 1), which are rather unstable due to the possibility of easy changes to these packages
 - The ones having many incoming dependencies and not many of outgoing ones (value of I is close to 0), therefore they are rather more difficult in modifying due to their greater responsibility



Example

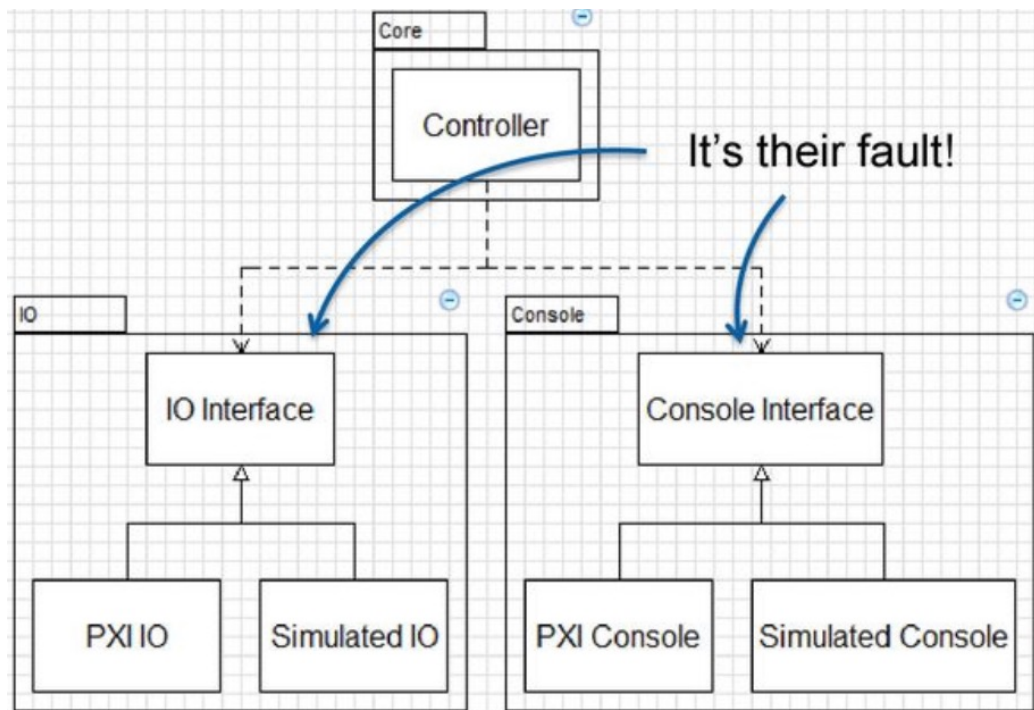


Stable-Dependencies Principle

- Change is inevitable
 - Some packages should be instable
 - Put the most volatile parts of your code in instable package
- High level architecture
 - Should be nonvolatile
 - Concentrated in the most stable packages



Example



Core Metrics

$Ca = 0$

$Ce = 1$

$I = 1$

Console Metrics

$Ca = 1$

$Ce = 0$

$I = 0$

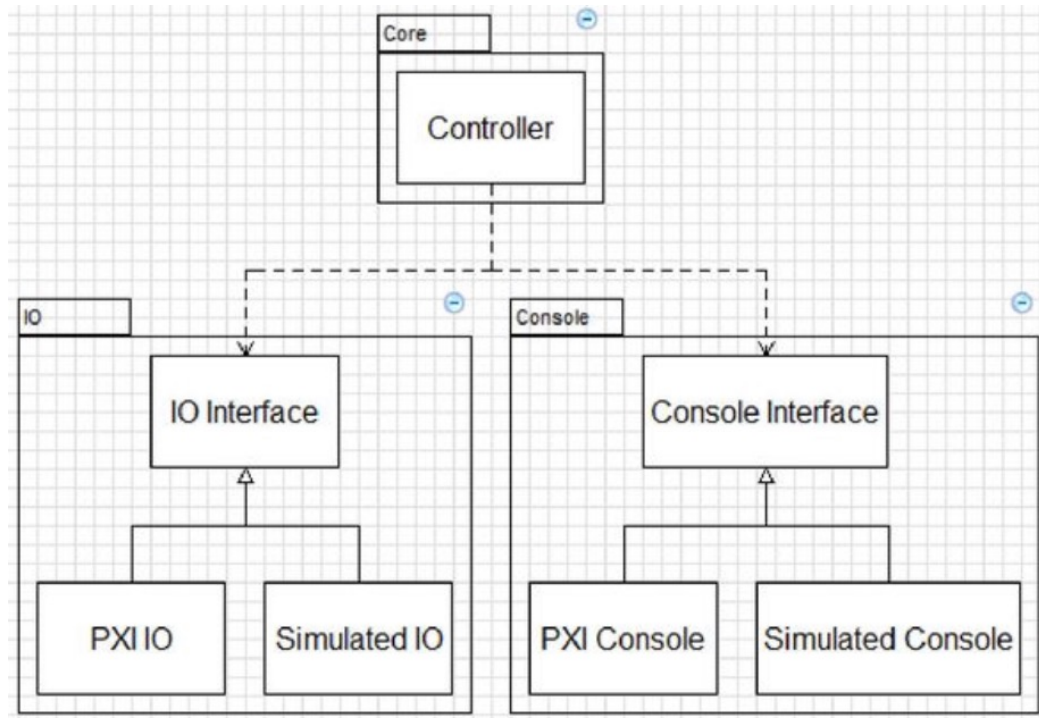


Abstractness (A)

- This metric is used to measure the degree of abstraction of the package and is somewhat similar to the instability
- $A = T_a / (T_a + T_c)$
 - T_a : number of abstract classes in a package
 - T_c : number of concrete classes in this package
- Packages that are stable (metric I close to 0) should also be abstract (metric A close to 1)
- Packages that are unstable (metric I close to 1) should consists of concrete classes (metric A close to 0)



Example



Core Metrics

$N_a = 0$

$N_c = 1$

$A = 0$

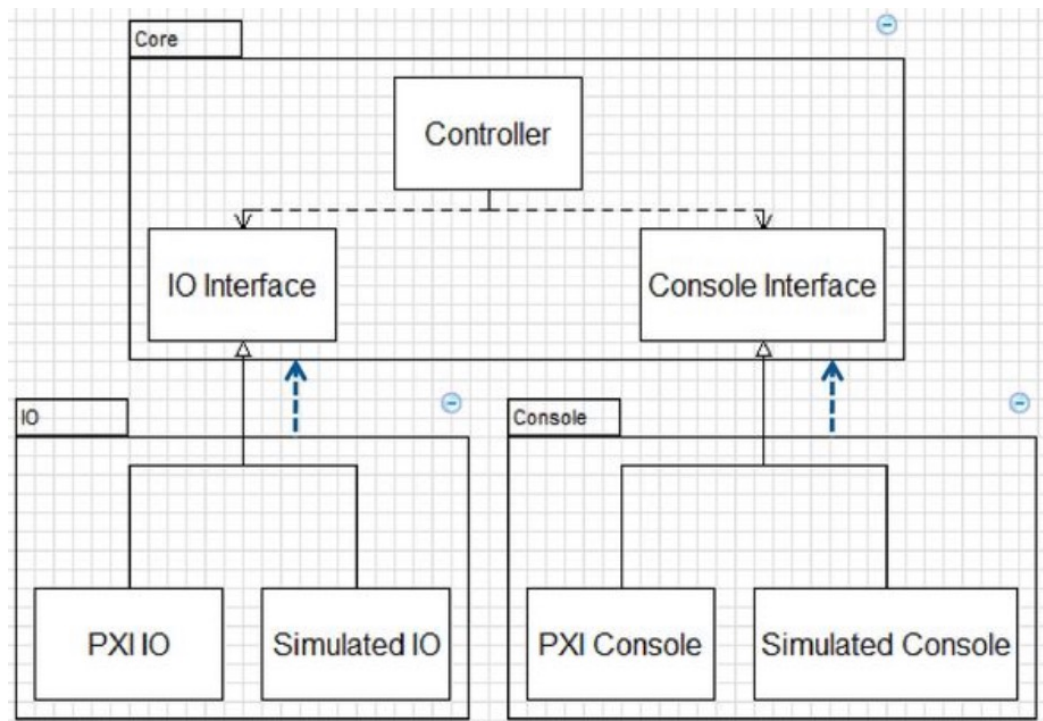
Console Metrics

$N_a = 1$

$N_c = 2$

$A = 0.5$

Better Solution



Core Metrics

$$I = 0$$

$$A = .667$$

Console Metrics

$$I = 1$$

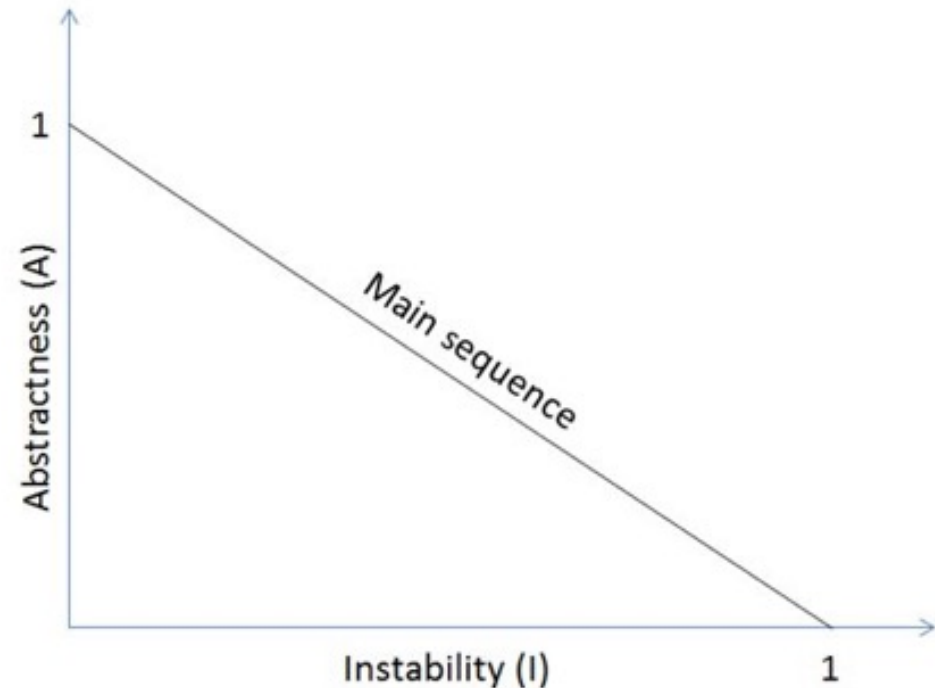
$$A = 0$$



Main Sequence

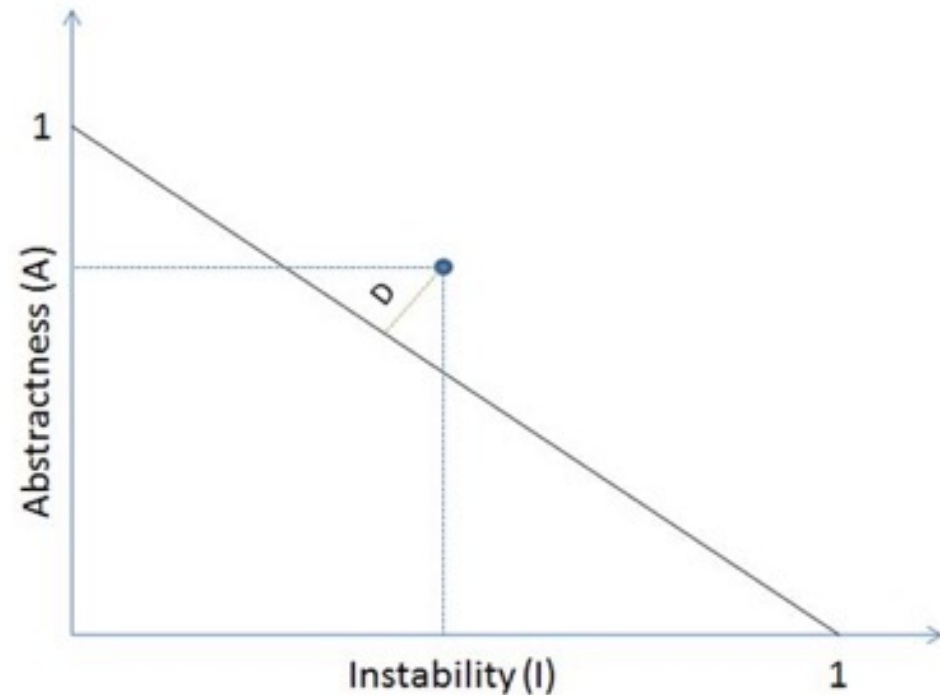
Đường trình tự chính

- Combining A and I formulates the existence of main sequence
- Classes that were well designed should group themselves around the graph end points along the main sequence



Normalized distance from main sequence (D)

- This metric is used to measure the balance between stability and abstractness and is calculated using the following formula
 - $D = |A + I - 1|$
 - If we put a given class on a graph of stability and abstractness, the distance from the main sequence will be proportional to the value of D.



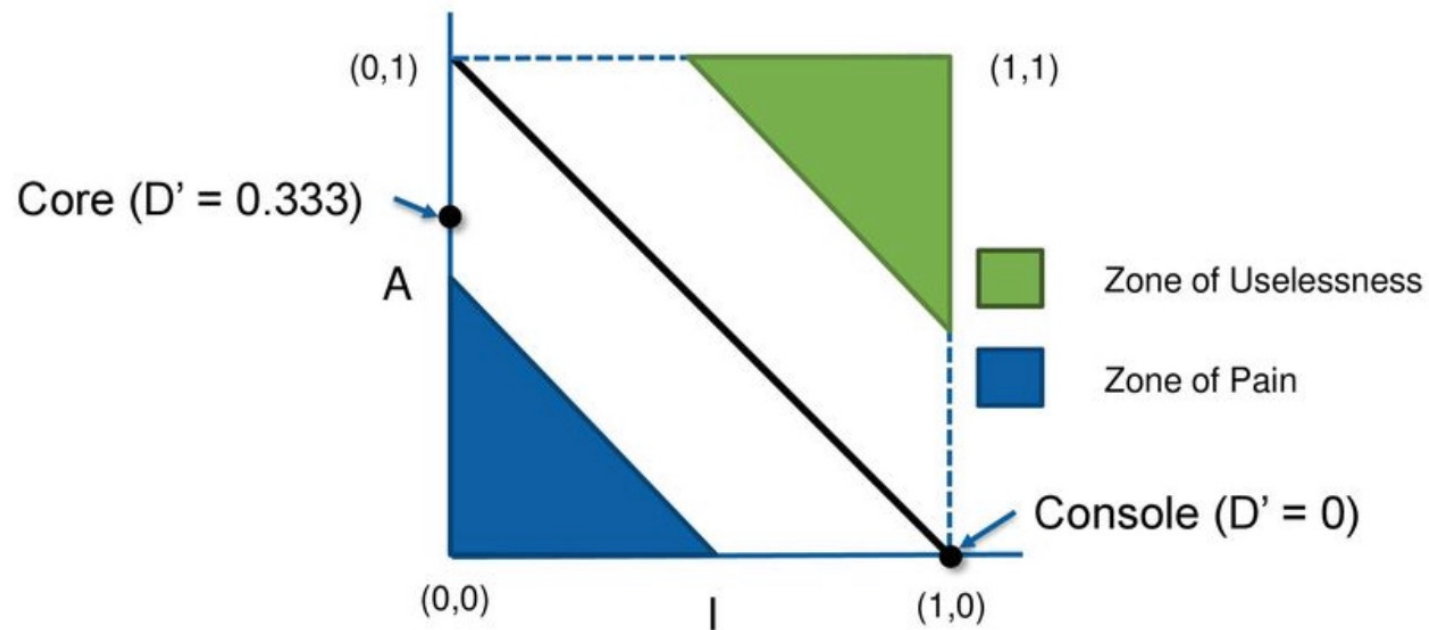
Benefits of Normalized distance from main sequence (D)

Ý nghĩa của chỉ số D

- The value of D should be as low as possible so that the components were located close to the main sequence
- $A = 0$ and $I = 0$, a package is extremely stable and concrete, the situation is undesirable because the package is very stiff and cannot be extended
- $A = 1$ and $I = 1$, rather impossible situation because a completely abstract package must have some connection to the outside, so that the instance that implements the functionality defined in abstract classes contained in this package could be created

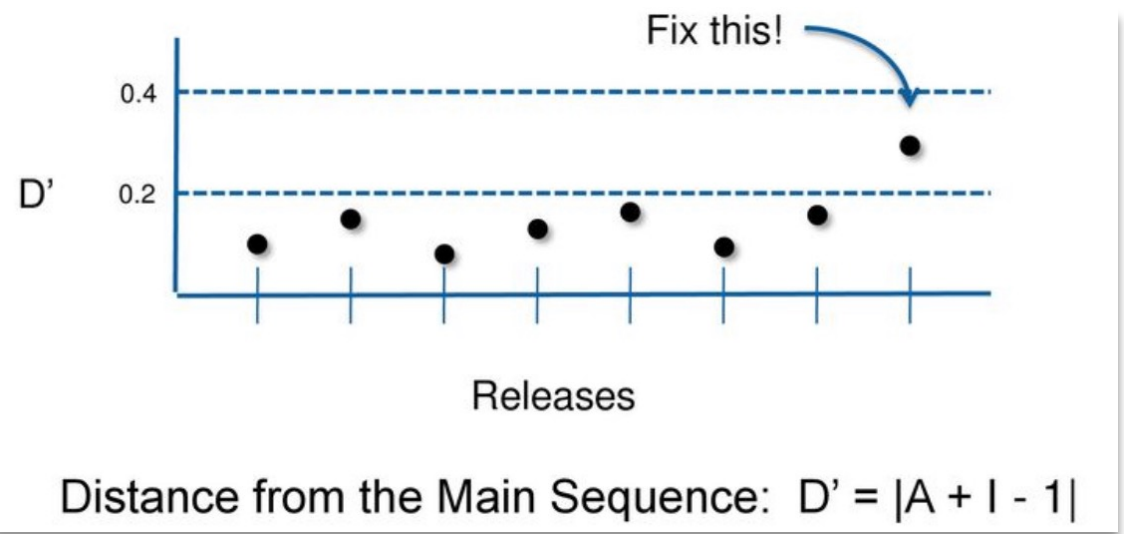


Example



Distance from the Main Sequence: $D' = |A + I - 1|$

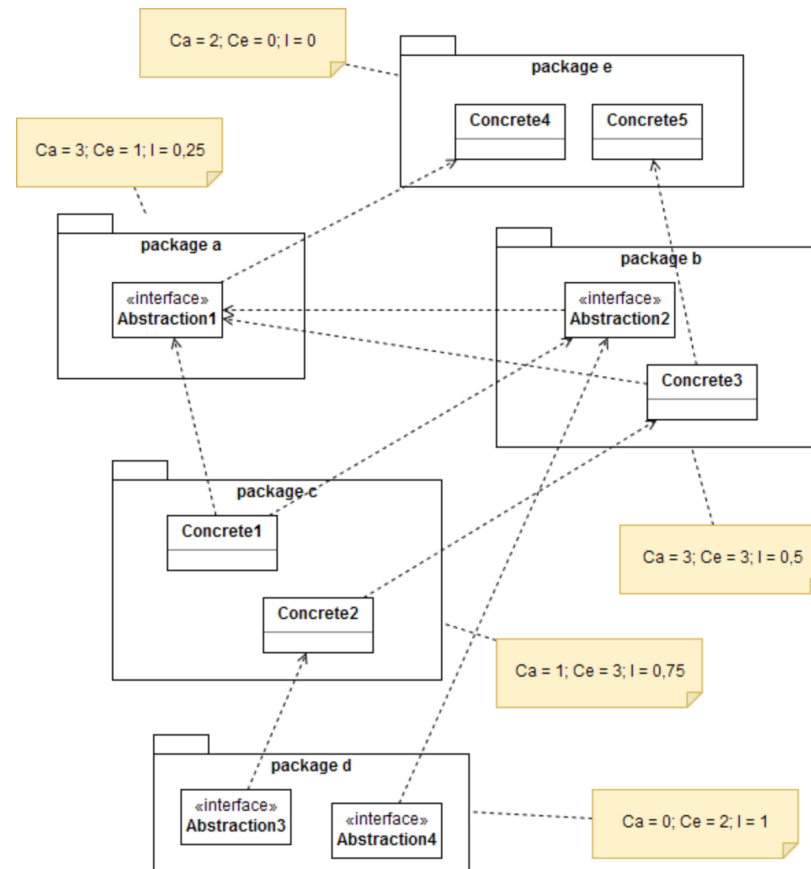
Track D



Summaries

- Students must understand the motivation of using software metrics to improve the quality of source code and also to improve the quality of software systems
- Give a general perspective about software metrics that are commonly used in software engineering
 - General metrics of lines of code
 - McCabe Cyclomatic Number
 - Halstead's metrics
 - Maintainability Index
 - OOP metrics







Software Quality Assurance and Testing

(end of lecture)

ONE LOVE. ONE FUTURE.